

T.D 2 : Récursivité en Python et structure de données immuables

I Complément sur la récursivité en Python

Exercice 1

1. Donner la complexité de la fonction suivante :

```
def somme_lst(t):
    if len(t) == 0:
        return 0
    return t[0] + somme_lst(t[1:])
```

2. Proposer une version de cette fonction n'utilisant pas de slice/tranche, mais utilisant les indices et une fonction récursive utilisant un paramètre nommé.
3. Proposer une version de cette fonction n'utilisant ni slice, ni paramètre nommé, mais utilisant les indices et une fonction récursive imbriquée dans une autre fonction. Quelle est la complexité dans ce cas ?
4. Appeler cette dernière fonction avec comme argument une liste Python contenant tout les nombres de 0 à 10000. Qu'observe-t-on ?
5. En utilisant la fonction `getrecursionlimit` du module `sys`, expliquer ce que vous avez observé.
6. En utilisant la fonction `setrecursionlimit` du module `sys`, effectuer le calcul.
7. Peut-on toujours expérimenter s'en sortir en fixant une limite plus élevée ?

Exercice 2

1. Ecrire une fonction `inverse_lst(lst, i=0, inv=[])` qui renvoie une liste Python contenant les éléments de `lst` mais dans l'ordre inverse. `inverse_lst([1, 2, 3, 4], i=0, inv=[])` renvoie `[4, 3, 2, 1]`.
2. Tester cette fonction sur `[1, 2, 3, 4]` puis `[5, 6, 7]`. Que remarque-t-on ?
3. Corriger la fonction en partant de : `inverse_lst(lst, i=0, inv=None)`

Exercice 3

La fonction `getsizeof(obj)` du module `sys` permet d'obtenir la taille d'un objet. Pour une liste Python, ce n'est pas le poids des objets auxquels la liste fait référence mais le poids des références.

1. Ecrire une fonction déterminant la taille d'un objet qui est un objet dont `getsizeof(obj)` détermine correctement la taille, ou un tuple, ou une liste. Dans les deux derniers cas, pouvant contenir récursivement ce type d'objets. On pourra utiliser la fonction `isinstance(obj, typ)`. On ne prendra pas en compte le fait qu'il soit possible de faire référence plusieurs fois au même élément.
2. Modifier la fonction pour qu'elle traite correctement le cas où il y a plusieurs références à un même élément. On pourra utiliser la fonction `id`, pour identifier un objet.
3. Modifier la fonction pour qu'elle traite également les dictionnaires.

II Structures de données immuables

Un objet immuable, en programmation orientée objet et fonctionnelle, est un objet dont l'état ne peut pas être modifié après sa création. Des exemples d'objets immuables/immuables en Python sont les `int`, `float`, `str` ou encore les **tuple**.

Les structures de données immuables sont particulières utilisées dans le cadre de la programmation fonctionnelle. Nous nous intéresserons en particulier aux listes récursives implémentées de façon immuable qui sont en général le type « conteneur de base » des langages fonctionnels.

Exercice 4

On cherche à faire une implémentation immuable du T.A.D liste vu en cours.
On a déjà le début d'implémentation suivant :

```
def lst_vide():
    return ()

def ajouter(elm, lst):
    return (elm, lst)
```

1. Poursuivre l'implémentation en donnant une définition cohérente des fonctions **tete** et **reste**. En utilisant les fonctionnalités du langage Python qui vous semblent pertinent traiter le cas de la liste vide.
2. Donner la complexité en temps des quatre opérations sur les listes.
3. Ecrire une fonction permettant de déterminer la longueur de la liste.
4. Ecrire une fonction qui prend en paramètre une liste Python `lstp` et renvoie une liste au sens du cours.
5. Ecrire une fonction permettant de déterminer la somme des éléments de la liste.
6. Ecrire une fonction qui affiche une liste (avec l'élément en tête à gauche.)
7. Ecrire une fonction prenant en entrée une liste et renvoyant une liste contenant ces éléments dans l'ordre inverse.

Exercice 5

1. A partir de deux listes immuables telles vu dans l'exercice précédent, donner une version objet et immuables des files. Utiliser les tests déjà écrits pour les files.
2. Donner la complexité en temps des différentes méthodes de ces files.

Exercice 6

Dans cet exercice, on reprend l'implémentation chaînées des listes vu dans l'exercice 4.

1. Ecrire une fonction qui renvoie l'avant dernier élément d'une liste.
2. Ecrire une fonction qui renvoie le nombre d'occurrences d'un élément donné dans une liste.
3. Ecrire une fonction qui renvoie l'élément maximal d'une liste.
4. Ecrire une fonction qui prend en entrée deux listes triées dans l'ordre croissant et renvoyant l'intersection de ces deux listes. La complexité devra être linéaire.
5. Même question pour l'union.
6. Ecrire une fonction prenant entrée deux listes triées dans l'ordre croissant et renvoyant une nouvelle liste contenant les éléments de ces deux listes dans l'ordre croissant et avec les mêmes multiplicités. La complexité devra être linéaire.
7. Ecrire une fonction réalisant le tri par insertion, d'une liste. Quelle est sa complexité ?
8. Ecrire une fonction réalisant le tri par sélection, d'une liste. Quelle est sa complexité ?
9. Ecrire une fonction réalisant le tri fusion (si vous l'avez déjà vu) d'une liste.